

Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Laboratorio 2 - Arreglos y excepciones

Estudiantes:
Emily Lucila Menchú Menchú - 20472
Daniel Alfredo Nájera de Paz - 26961

Fecha: 07/09/2026

Análisis

Requisitos funcionales

- Crear y reemplazar el festival actual.
- Inicializar cada festival sin escenarios ni artistas.
- Configurar escenarios por posición (Las posiciones serán de 1 a 5, para no confundir el usuario).
- Consultar todos los escenarios.
- Consultar un escenario por código (decidimos hacerlo por código o nombre para que sea más fácil para el usuario consultar un escenario, ya que las posiciones/índices en una lista pueden ser confusos para el usuario).
- Modificar capacidad y estado de los escenarios.
- Retirar un escenario y dejar la posición en null.
- Registrar artistas sin códigos repetidos.
- Consultar artistas.
- Buscar artista por código o nombre.
- Modificar artista (como extra se puede modificar el escenario en el que participará).
- Cancelar participación de artista eliminándolo de la lista.
- Calcular escenarios configurados.
- Calcular espacios disponibles.
- Encontrar el escenario con mayor capacidad.
- Calcular total de artistas registrados.
- Encontrar el artista con mayor duración de presentación.
- Encontrar el artista con mayor asistencia.
- Calcular el promedio de duración.
- Manejar colecciones vacías.
- Manejar InputMismatchException.
- Generar y manejar IllegalArgumentException.
- Utilizar al menos un finally.
- Continuar la ejecución después de errores manejados con try/catch.
- Mostrar un menú con 13 opciones.

Clases y motivos:

Modelo:

- **Artista:** Esta clase servirá como modelo de los datos que el Artista guarda, también validará los datos que se guarden en el objeto.
- **Escenario:** Esta clase guardará los datos de los escenarios y validaciones de los datos que guarda.
- **Festival:** Esta clase guarda los datos del festival activo.

Controlador

- **ControladorFestival:** Esta clase gestionará la comunicación de las clases del modelo con la vista.

Vista

- **VistaFestival:** Esta clase tendrá métodos que interactuarán con el usuario, pidiendo y mostrando datos.
- **DriverProgram:**
- **Main:** Esta clase inicializará el programa, será el punto de entrada al sistema.

Relaciones entre clases:

- Main tendrá una relación de dependencia con ControladorFestival, porque crea una instancia del controlador y utiliza su método iniciar() para comenzar el programa.
- ControladorFestival tendrá una relación de agregación con Festival, porque tiene una instancia del festival actual.
- ControladorFestival tendrá una relación de agregación con Escenario, porque tiene un arreglo básico de cinco objetos Escenario. Las posiciones que todavía no tengan un escenario serán null.
- ControladorFestival tendrá una relación de agregación con Artista, porque contiene y administra un ArrayList<Artista> con los artistas registrados en el festival.
- ControladorFestival tendrá una relación de asociación con VistaFestival, porque tiene una referencia a la vista y usará sus métodos para interactuar con el usuario.

¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?

La propiedad escenarios de la clase ControladorFestival debe implementarse utilizando un arreglo básico. Este arreglo almacenará objetos de tipo Escenario y tendrá un tamaño fijo de 5 posiciones, debido a que se contempla un máximo de cinco escenarios para el festival.

¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

La propiedad artistas de la clase ControladorFestival debe implementarse utilizando un ArrayList. Este almacenará objetos de tipo Artista, ya que la cantidad de artistas puede variar durante la ejecución del programa y se requiere poder agregar, buscar, modificar y eliminar elementos dinámicamente.

¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

Los atributos de las clases deben tener visibilidad private, con el objetivo de aplicar el principio de encapsulamiento y evitar modificaciones directas desde otras clases. Los constructores, métodos de acceso, métodos de modificación y demás operaciones que necesiten ser utilizadas desde otras clases deben ser public. Los métodos auxiliares destinados únicamente a validaciones internas pueden establecerse como private.

¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

Los valores iniciales de los objetos serán proporcionados mediante los constructores de cada clase, los cuales permitirán establecer los atributos necesarios al momento de crear cada objeto.

Antes de realizar modificaciones deberán validarse principalmente los valores numéricos. La duración de una presentación y la capacidad máxima de un escenario deberán ser mayores que cero, mientras que la cantidad de asistentes deberá cumplir con los valores permitidos. También se debe comprobar que datos importantes, como códigos y nombres, sean válidos y no estén vacíos.

¿Cómo determinará si una posición del arreglo contiene un escenario o contiene null?

Para determinarlo se deberá recorrer el arreglo de escenarios y revisar el contenido de cada posición. Si una posición contiene null, significa que está disponible y que todavía no tiene un escenario asignado. Si la posición es diferente de null, significa que contiene un objeto de tipo Escenario. Esta comprobación también permite evitar errores al intentar utilizar una posición vacía.

¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

Las operaciones se realizarán recorriendo el ArrayList de artistas y utilizando el código del artista como identificador.

Para realizar una búsqueda, se comparará el código proporcionado con el código de cada artista hasta encontrar una coincidencia. Para modificar, primero se localizará al artista y posteriormente se actualizarán los atributos correspondientes. Para eliminar, se localizará al artista por su código y después se removerá del ArrayList. En todos los casos se deberá verificar primero que el artista exista.

¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Las principales excepciones pueden producirse durante el ingreso de datos por parte del usuario, especialmente cuando se introduce un tipo de dato diferente al solicitado. En estos casos puede presentarse una `InputMismatchException`. También puede producirse una `NumberFormatException` al intentar convertir un texto que no representa un número válido. Asimismo, se debe considerar la posibilidad de una `NullPointerException` al intentar utilizar una posición vacía del arreglo y una `IndexOutOfBoundsException` al acceder a una posición que se encuentre fuera de los límites permitidos. Estas situaciones deben prevenirse mediante las validaciones correspondientes.

El bloque try-catch se utilizará principalmente en la clase `VistaFestival`, debido a que esta se encarga de recibir los datos ingresados por el usuario y es donde existe mayor posibilidad

de entradas incorrectas. El bloque finally lo usaremos principalmente en la vista, específicamente en los métodos de pedirInt() y pedirFloat() después de cada intento de lectura, tanto si fue correcto como si hubo una excepción, el finally se asegurará de limpiar el buffer con sc.nextLine().

Tabla de atributos

#	Tipo	Nombre	Visibilidad	Motivo	Clase
1	String	codigoArtista	private	El codigo personal de cada artista	Artista
2	String	nombreArtistico	private	El nombre artistico del artista	Artista
3	String	generoMusical	private	El genero musical del artista	Artista
4	float	duracionPresentacion	private	La duración de la presentacion del artista	Artista
5	int	asistentes	private	Cuantos asistentes va a atraer el artista	Artista
6	String	escenario	private	El código del escenario donde se va a presentar el artista	Artista
7	String	codigoEscenario	private	El código del escenario	Escenario
8	String	nombre	private	El nombre del escenario	Escenario
9	String	ubicación	private	La ubicación del escenario	Escenario
10	int	capacidadMax	private	La capacidad máxima que soporta el escenario	Escenario
11	boolean	estado	private	Si el escenario está ocupado o libre	Escenario
12	Escenario[5]	escenarios	private	Los escenarios que va a usar el festival	ControladorFestival
13	Festival	festival	private	El festival	ControladorFestival
14	ArrayList<Artista>	artistas	private	Todos los artistas del festival	ControladorFestival

15	VistaFestival	vista	private	Guarda la vista	Controlador Festival
16	String	nombreFestival	private	El nombre del festival	Festival
17	String	codigoFestival	private	El código del Festival	Festival
18	String	nombreCoordinador	private	Nombre del coordinador del festival	Festival
19	Scanner	sc	private	Guarda el escáner	VistaFestival

Tabla de Métodos

#	Tipo de retorno	Nombre	Visibilidad	Parámetro	Motivo	Clase
1	-	Artista	public	codigoArtista: String, nombreArtístico:String, generoMusical: String, duracionPresentacion: float, asistentes: int, escenario: String	Constructor de la clase artista que asigna los valores iniciales del artista.	Artista
2	String	getCodigoArtista	public	-	Obtiene el código del artista	Artista
3	String	getNombreArtístico	public	-	Obtiene el nombre artístico	Artista
4	String	getGeneroMusical	public	-	Obtiene género musical	Artista
5	float	getDuracionPresentacion	public	-	Obtiene duración de presentación	Artista

6	int	getAsistentes	public	-	Obtiene asistentes	Artista
7	String	getCodigoEscenario	public	-	Obtiene el código del escenario al que se asignó	Artista
8	void	setNombreArtístico	public	nombreArtístico: String	Modifica el nombre artístico	Artista
9	void	setGeneroMusical	public	generoMusical: String	Modifica el género de musical	Artista
10	void	setDuracionPresentacion	public	duracionPresentacion: float	Modifica la duración de la presentación	Artista
11	void	setAsistentes	public	asistentes: int	Modifica la cantidad de asistentes	Artista
12	void	setCodigoEscenario	public	codigo: String	Modifica el escenario en donde se presentará el artista	Artista
13	boolean	validarDuracin	private	numero: float	Valida que el numero ingresado sea mayor a 0. Lanza IllegalArgumentException	Artista
14	String	toString	public	-	Devuelve la información del artista como String.	Artista
15	boolean	validarAsistentes	private	numero: int	Valida que el numero ingresado sea positivo. Lanza IllegalArgumentException	Artista
16		Escenario	public	codgoEscenario: String, nombre: String, ubicación: String, capacidad Max: int,	Constructor de la clase escenario que asigna los valores iniciales	Escenario

				estado: boolean		
17	String	getCodigo Escenario	public	-	Obtiene el código del escenario	Escenario
18	String	getNombr e	public	-	Obtiene el nombre del escenario	Escenario
19	String	getUbicaci ón	public	-	Obtiene el ubicación del escenario	Escenario
20	int	getCapaci dadMax	public	-	Obtiene la capacidad máxima	Escenario
21	boolean	getEstado	public	-	Obtiene el estado del escenario	Escenario
22	void	setCapaci dadMax	public	max: int	Modifica la capacidad máxima	Escenario
23	void	setEstado	public	estado: boolean	Modifica Estado	Escenario
24	boolean	validarCap acidadMax	private	num: int	Valida que el número sea mayor a 0. Lanza IllegalArgumentException	Escenario
25	String	toString	public	-	Retorna la información del escenario como String	Escenario
26	-	Festival	public	nombreFes tival: String, codigoFesti val: String, nombreCoo rdinador: String	Constructor de Festival que asigna valores iniciales	Festival
27	String	toString	public	-	Retorna la información del Festival como String	Festival
28	-	Controlado rFestival	public	-	Constructor de la clase ControladorFestival y dentro creará y	ContoladorFe stival

					asignará valores de atributos	
29	void	iniciar	public	-	Inicia el programa	ControladorFestival
30	void	nuevoFestival	private	-	Pide a la vista datos para asignar los valores al festival	ControladorFestival
31	int	mostrarMenu	private	-	A través de la vista muestra el menú y retorna la opción ingresada por el usuario	ControladorFestival
32	void	configurarEscenario	private	-	Crea un nuevo escenario en una posición específica	ControladorFestival
33	void	consultarEscenarios	private	-	A través de la vista muestra todos los escenarios	ControladorFestival
34	void	consultarEscenario	private	-	Busca un escenario por su posición. Puede lanzar NullPointerException e IndexOutOfBoundsException	ControladorFestival
35	void	modificarEscenario	private	-	Modifica el escenario por su posición. Puede lanzar NullPointerException e IndexOutOfBoundsException	ControladorFestival
36	void	retirarEscenario	private	-	Retira el escenario de la lista asignando null en donde solía estar por su posición. Puede generar NullPointerException	ControladorFestival

					e IndexOutOfBoundsException	
37	void	registrarArtista	private	-	Crea un nuevo artista y lo agrega a la lista. Pero primero se recorrerá la lista para verificar que el código no exista ya.	ControladorFestival
38	void	consultarArtistas	private	-	Gestiona la muestra de todos los artistas	ControladorFestival
39	void	buscarArtista	private	-	Busca a un artista por su código o nombre.	ControladorFestival
40	void	modificarArtista	private	-	Modifica un artista por su código o nombre	ControladorFestival
41	void	cancelarParticipación	private	-	Elimina al artista de la lista por código o nombre	ControladorFestival
42	void	mostrarReporteFestival	private	-	Gestiona la llamada de métodos para hacer los cálculos y mostrarlos a través de la vista	ControladorFestival
43	int	calcularTotalEscenarios	private	-	Calcula el total de escenarios	ControladorFestival
44	Escenario	calcularEscenarioMayorCapacidad	private	-	Calcula el escenario con mayor capacidad de asistentes	ControladorFestival
45	int	calcularTotalArtistas	private	-	Calcula el total de artistas registrado	ControladorFestival
46	Artista	getArtistaMayorDuracion	private	-	Busca el artista con mayor duración	ControladorFestival

47	Artista	getArtista MayorAsis tentes	private	-	Busca el artista con mayor cantidad de asistentes	ControladorF estival
48	float	calcularPr omedioDur ación	private	-	Calcula Promedio de duraciones de presentaciones	ControladorF estival
49	int	mostrarMe nu	public	String: msj	Muestra el menú y retorna la opción ingresada por el usuario	VistaFestival
50	String	pedirString	public	-	Pide al usuario un String	VistaFestival
51	int	pedirInt	public	-	Pide al usuario un int. Puede lanzar excepción InputMismatchExcepti on	VistaFestival
52	float	pedirFloat	public	-	Pide al usuario un float. Puede lanzar excepción InputMismatchExcepti on	VistaFestival
53	void	mostrarMe nsaje	public	String:msj	Le muestra al usuario un mensaje	VistaFestival
54	void	main	public	args: String[]	Método inicializador del programa	Main